AIKernel Phase-1 Paper 02: VFS Architecture and Semantic Storage Model

Virtual File System and Capability Abstraction for Deterministic AIOS Governance

Takuya Sogawa

2026-05-24

AIKernel Phase-1 Paper 02: VFS Architecture and Semantic Storage Model

Virtual File System and Capability Abstraction for Deterministic AIOS Governance

Author: Takuya Sogawa
ORCID: https://orcid.org/0009-0009-7499-2595
Version: v0.2.0
Publication date: 2026-05-24
DOI: https://doi.org/10.5281/zenodo.20307891

Canonical language: English
Companion translation: Japanese
Series position: AIKernel / AIOS Phase-1 Specification Series, Part I-2: Knowledge Substrate
Target: AIKernel.NET Core / VFS
Proposed namespace: `AIKernel.Abstractions.Vfs`
Stability: Experimental / Non-normative

Abstract

Autonomous AI systems based on Large Language Models (LLMs) increasingly interact with files, repositories, object stores, databases, logs, snapshots, and other persistence surfaces. When storage access is represented as ordinary paths or opaque strings, the governance layer cannot reliably distinguish what can be read, written, deleted, queried, or replayed before an operation reaches the provider. As a result, unsupported operations are often discovered only through late runtime failures, while unsafe operations may escape into mutable infrastructure before they are evaluated.

This paper defines the **VFS Architecture and Semantic Storage Model** as the second knowledge-substrate paper in the AIKernel / AIOS Phase-1 specification series. It extends the ROM model defined in Paper 01 by specifying the storage boundary through which ROM documents, context snapshots, conversation histories, and replay artifacts are persisted and recovered.

The central proposal is a **Formal Storage Capability Theory**. Storage access is not modeled as a set of best-effort runtime methods that may throw `NotSupportedException`; it is modeled as a set of explicit **Capability Contracts** exposed through the type system. A session that does not implement a capability does not merely reject the operation late. The operation is considered **non-routable**: it has no valid transition path through the VFS boundary.

The model introduces semantic storage paths, bounded storage topology, type-level truthfulness, deterministic read-only projection for replay, and fail-closed capability gating. Together, these mechanisms define

a storage substrate that supports deterministic governance, audit-grade replay semantics, and safe persistence of AIKernel knowledge assets.

Keywords

AIKernel; AIOS; Virtual File System; VFS; Semantic Storage; Capability Contracts; Type-Level Truthfulness; Non-Routable Transition; Bounded Storage Topology; Deterministic Replay; Read-Only Projection; Audit-Grade Execution; Fail-Closed; Knowledge Substrate; AI Operating System

---

1. Introduction

AI agents increasingly operate over persistent state. They read project files, write generated artifacts, inspect logs, load memory snapshots, retrieve knowledge documents, and interact with external storage providers. In conventional applications, storage errors are often treated as ordinary implementation concerns. In autonomous AI systems, however, storage access becomes part of the governance boundary.

The problem is not only whether a file exists. The deeper problem is whether the system can determine, before execution, which storage transitions are admissible, which are prohibited by policy, and which are physically impossible because the provider or session does not possess the relevant capability.

AIKernel therefore treats storage not as a transparent utility layer but as a governed semantic boundary. The Virtual File System (VFS) provides a common abstraction for storage providers while preserving the distinction between read, write, delete, navigation, query, and replay capabilities. This makes storage operations visible to the governance layer before side effects occur.

This paper defines the VFS Architecture and Semantic Storage Model as **Paper 02** in the AIKernel / AIOS Phase-1 specification series. Paper 01 defines ROM documents and knowledge snapshots as immutable knowledge units. Paper 02 defines the storage boundary through which those knowledge units are persisted, retrieved, projected, and replayed.

The main principle is simple:

> Storage capabilities must be explicit before execution. Unsupported transitions must be non-routable, not merely rejected after dispatch.

2. Problem Statement: Late Runtime Failure and Storage Entropy

When AI systems interact with file systems and external storage providers, several structural problems emerge.

2.1 Late Runtime Failure

Different storage backends support different operations. A local file system may support reading, writing, deletion, directory navigation, and metadata updates. An object store may support only a subset of these operations. A replay provider may intentionally support only read-only observation.

In conventional abstractions, unsupported operations are often represented as methods that exist but throw exceptions at runtime. For deterministic AI governance, this is too late. If a delete operation reaches a provider before the governance layer can determine whether deletion is physically supported and policy-authorized, the boundary is already too weak.

2.2 Opaque Capabilities

LLMs can propose storage operations as action candidates: open a file, rewrite a document, delete a cache, scan a directory, or query a repository. If storage capabilities are opaque, the Policy Decision Point (PDP) cannot reliably determine whether the proposed action is admissible before dispatch.

Opaque capabilities force the system to discover authority by trial and error. AIKernel rejects this pattern. Authority must be visible as a contract before execution.

2.3 Storage Entropy and Side-Effect Surfaces

Unrestricted mutable storage introduces uncontrolled side-effect surfaces. Treating paths and URIs as ordinary strings loses the semantic meaning of the target: whether it belongs to a ROM zone, a replay archive, a mutable workspace, a secret store, or an external side-effect boundary.

This ambiguity increases storage entropy. A deterministic governance system must reduce that entropy by assigning storage entries to governed semantic zones and by making capability boundaries explicit.

3. Design Goals

The VFS model is designed around the following goals.

1. **Capability-Based Interface Segregation.** Read, write, delete, navigation, and query operations must be represented by explicit capability-bearing interfaces rather than unsupported placeholder methods.
2. **Pre-dispatch Verification.** Required capabilities must be checked before side effects occur. If the authority or route does not exist, the operation must fail closed.
3. **Provider Abstraction.** Physical storage differences must be hidden behind provider adapters without hiding their capabilities.
4. **Semantic Storage.** Paths and entries must preserve governance-relevant meaning, including zones, provenance, policy constraints, and replay status.
5. **Deterministic Replay Support.** Past execution contexts and knowledge assets must be recoverable through stable references and read-only replay projection.

4. Scope and Boundary

This paper is responsible for **storage boundary governance** within the AIKernel / AIOS Phase-1 architecture. It does not define ROM identity itself; that is the role of Paper 01. It does not define full pre-inference admissibility policy; that is the role of Paper 03. It does not define trajectory-level runtime convergence; that is the role of Paper 04.

Layer / Paper	Entropy Role in Governance
ROM (Paper 01)	Knowledge entropy reduction
VFS (Paper 02)	**Storage boundary governance**
Governance (Paper 03)	Execution admissibility
Trajectory (Paper 04)	Runtime boundedness
Semantic Compiler (Phase 2)	Semantic entropy reduction

The VFS layer therefore answers a narrower question:

> Given a storage provider, a session, a path, and an operation, can the operation be routed safely through the storage boundary before any side effect occurs?

5. Semantic Storage Model

The VFS architecture in AIKernel is not merely a byte store. It is the semantic storage substrate for governed knowledge assets, including ROM documents, snapshots, replay records, and context materials.

Traditional storage and AIKernel semantic storage differ as follows.

Traditional Storage	AIKernel Semantic Storage
Byte-addressed	Semantically addressed
Opaque paths	Governed semantic paths
Mutable file system	Replayable substrate
Provider-specific behavior	Capability-declared behavior
Late runtime failure	Pre-dispatch fail-closed evaluation

A governed semantic path is not merely a string. It carries or resolves to governance-relevant meaning: namespace, provider identity, zone, provenance, policy context, and replay status. This allows AIKernel to reason about storage transitions as governed state transitions rather than arbitrary file operations.

6. Formal Storage Capability Theory

6.1 VFS Session State

The state of a VFS session is defined as:

```
S_vfs = <Provider, Auth, Cap, Sigma>
```

where:

- `Provider` is the physical or logical storage provider identity.
- `Auth` is the authenticated session context.
- `Cap` is the set of static capabilities exposed by the session.
- Σ is the set of observable storage entry states.

6.2 Semantic Admissibility Boundary

A capability is not merely a permission flag. In AIKernel, it is an execution topology constraint. It determines whether a transition can be routed through the VFS boundary at all.

Let `C_allowed` be the capability space permitted by the system policy. A VFS session must satisfy:

```
Cap(S_vfs) ⊆ C_allowed
```

The system distinguishes three operation classes.

Operation class	Meaning
Admissible	The capability exists and the operation may proceed to policy evaluation.
Prohibited	The operation is physically possible but denied by policy.
Non-routable	The session lacks the capability, so no valid route exists.

This distinction is important. A prohibited operation is rejected by governance policy. A non-routable operation is rejected because the architecture exposes no transition path.

6.3 Non-Routable Transition Model

Unsupported operations are not treated as late runtime failures. They are treated as physically or architecturally unreachable transitions.

For an operation `op`, if no route exists, the state transition function immediately returns a fail-closed halt state:

```
Route(op) = ∅ => delta_op(S_vfs, path) = S_halt
```

This is not a guarantee about a particular programming language runtime. It is an architectural requirement: unsupported operations must be rejected before provider dispatch and before side effects occur.

6.4 Bounded Storage Topology

The VFS is governed as a bounded storage topology, not as an unstructured hierarchy of paths.

Topology zone	Semantic purpose
Immutable zones	ROM and other immutable knowledge assets.
Replay-only zones	Historical records used for audit and reconstruction.
Mutable zones	Runtime workspaces with explicitly granted write authority.
Query zones	Searchable or index-backed regions with bounded query semantics.
External side-effect boundaries	Regions where writes affect external systems.

Each zone defines a different governance profile. For example, replay-only zones may expose read capabilities but must not expose write or delete capabilities during replay.

6.5 Type-Level Truthfulness

The core invariant of the VFS architecture is **Type-Level Truthfulness**:

```
Implemented Interface ⇔ Physically Realizable Operation
```

A provider must not implement an interface for an operation it cannot actually perform. Implementing a method and then throwing `NotSupportedException` for normal unsupported capability cases is a contract violation in this model.

The purpose is not stylistic purity. The purpose is to make the storage boundary truthful to the governance layer.

7. Governance and Security Considerations

The VFS layer collaborates with AIKernel governance to enforce storage safety before side effects occur.

Threat	Mitigation
Unauthorized mutation	Capability gating and pre-dispatch verification.
Replay contamination	Deterministic read-only projection.
Path traversal	Governed semantic paths and bounded topology.
Provider spoofing	Provider identity contracts and signed manifests.
Capability confusion	Type-level truthfulness and interface segregation.

If an LLM proposes a deletion command against a session that does not expose `IDeletableVfsSession`, the operation is non-routable. It is not sent to the provider and is not left to fail later. This turns storage authority into an observable governance fact.

8. Determinism and Replayability

Deterministic replay requires storage state to be projected safely. During replay, the purpose is not to perform the original side effects again. The purpose is to reconstruct the evidence and context under which the original decision was made.

AIKernel therefore defines **Deterministic Read-Only Projection**. In replay mode, the original storage provider is remounted or represented through a provider that exposes only read and observation capabilities.

Runtime execution	Replay execution
Mutable transitions may be allowed.	Observational transitions only.
Side effects may occur under governance.	Side effects are quarantined.
Provider capabilities reflect live authority.	Provider capabilities reflect replay authority.

This prevents replay from modifying production data, altering logs, or creating new external side effects.

9. Implementation Architecture

The model maps naturally to capability-bearing interfaces in `AIKernel.Abstractions.Vfs`.

- `IVfsProvider`: creates authenticated sessions and declares provider identity.
- `IVfsEntryInfo`: represents common entry metadata.
- `IReadableVfsFile`: represents readable file content.
- `IWritableVfsFile`: represents writable file content.
- `INavigableVfsDirectory`: represents directory enumeration capability.
- `IReadableVfsSession`: exposes read operations.
- `IWritableVfsSession`: exposes write operations.
- `IDeletableVfsSession`: exposes deletion operations.
- `IQueryableVfsSession`: exposes bounded query operations.

A provider may expose multiple capabilities, but it must expose only those capabilities it can truthfully realize.

10. Limitations

This architecture has several limitations.

1. **No independent distributed-consistency guarantee.** VFS defines semantic storage boundaries. It does not, by itself, solve distributed transactions, object-store consistency, or network partition behavior.
2. **Capability granularity and access-control boundary.** The initial VFS model intentionally exposes coarse-grained capabilities such as read, write, delete, navigate, and query. Fine-grained authorization, such as allowing writes only under a specific directory or only under a given user attribute, belongs to higher-level policy evaluation. In this sense, ABAC and XACML-style policy systems are treated as PDP/PEP-layer mechanisms, while VFS provides truthful session capabilities and semantic storage facts consumed by those policies.
3. **Provider honesty requirement.** The model assumes providers honestly expose their capabilities. Provider identity contracts, signed manifests, testing, and attestation are needed for stronger assurance.
4. **Semantic path design.** The model depends on disciplined path and namespace design. Poorly designed namespaces can still produce confusing governance boundaries.

11. Relationship to Other Phase-1 Papers

This paper defines the physical and semantic storage boundary for AIKernel's knowledge and data layer.

- **Paper 01: ROM Format and Knowledge Snapshot Model** defines the governed knowledge units stored and retrieved through VFS.
- **Paper 03: Pre-Inference Admissibility Governance** evaluates whether proposed storage operations are admissible before execution.
- **Paper 04: Trajectory Governance Model** assumes storage operations are already bounded by VFS and governance policies.
- **Paper 05 / Paper 07: Execution and Implementation** use VFS abstractions for execution state persistence, conversation snapshots, and replay archives.

12. Conclusion

The AIKernel VFS Architecture defines storage access as an explicit capability boundary rather than as a best-effort runtime abstraction. By requiring type-level truthfulness, capability-bearing interfaces, non-routable transitions, bounded storage topology, and deterministic read-only replay projection, the model prevents storage side effects from escaping governance.

This paper establishes the storage side of the AIKernel knowledge substrate. Together with the ROM model defined in Paper 01, it provides the foundation on which later Phase-1 papers build admissibility governance, trajectory governance, execution control, delegation, implementation, and unified AIOS architecture.

References

1. Dennis, Jack B., and Earl C. Van Horn. "Programming Semantics for Multiprogrammed Computations." Communications of the ACM, vol. 9, no. 3, 1966, pp. 143-155. DOI: 10.1145/365230.365252.
2. Lampson, Butler W. "Protection." Proceedings of the Fifth Annual Princeton Conference on Information Sciences and Systems, 1971, pp. 437-443.
3. Levy, Henry M. Capability-Based Computer Systems. Digital Press, 1984.
4. Tanenbaum, Andrew S., and Herbert Bos. Modern Operating Systems. 4th ed., Pearson, 2014.
5. Hu, Vincent C., David Ferraiolo, Rick Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. Guide to Attribute Based Access Control (ABAC) Definition and Considerations. NIST Special Publication 800-162, 2014. DOI: 10.6028/NIST.SP.800-162.
6. OASIS. eXtensible Access Control Markup Language (XACML) Version 3.0. OASIS Standard, 22 January 2013.
7. Fowler, Martin. "Event Sourcing." martinfowler.com, 2005.
8. Chen, Peter M., and Brian D. Noble. "When Virtual Is Better Than Real." Proceedings of the 8th Workshop on Hot Topics in Operating Systems (HotOS VIII), 2001, pp. 133-138.
9. Sogawa, Takuya. "Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model." Zenodo, 2026. DOI: 10.5281/zenodo.20290614.
10. Sogawa, Takuya. "AIKernel Phase-1 Paper 01: ROM Format and Knowledge Snapshot Model." Zenodo, 2026. DOI: 10.5281/zenodo.20306539.